



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .

2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The **state space** means all the possible states that the agent can reach in the environment. In our grid game, different positions and directions of the agent are different states.

The **search tree** is created when the search algorithm starts from the initial state and explores different possible paths until it finds the goal.

So, simply:

State space = all possible states.

Search tree = the paths and states explored by the search algorithm.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The **reached** set keeps a record of the states that the agent has already visited.

This prevents the search algorithm from visiting the same state again and again. It also helps to avoid loops.

Without the **reached** set, DFS could keep going through the same states repeatedly and may waste a lot of time or even get stuck in a loop instead of finding the food.

This is important because search should explore new states instead of repeating the same ones.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS explores the **shallowest nodes first**, level by level. Because every action in our grid has the same cost, BFS will find the path with the smallest number of actions first. The lecture notes also state that BFS is optimal when the cost is **1** per step.

DFS works differently. It explores the **deepest path first**. Because of this, it may go far in a wrong or longer direction before finding the food.

In my simulation:

BFS → 89 steps
DFS → 608 steps

This clearly shows that BFS found much shorter paths, while DFS took long and winding routes.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS uses a lot of memory because it keeps many nodes in the frontier at the same time. The lecture notes explain that BFS has exponential space usage, around $O(b^d)$ in the summary, where b is the branching factor and d is the depth of the solution. This means the number of stored nodes can become very large when the grid becomes bigger.

DFS uses less memory because it mainly keeps one path and some unexplored choices. Its space complexity is given as $O(bm)$, where m is the maximum depth.

So, simply:

- **BFS** → uses more memory but can find the shortest path when all step costs are equal.
- **DFS** → uses less memory, but it can take very long and inefficient paths.

For a very large grid, BFS may run out of memory faster than DFS.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING